

Лабораторна робота №4

MongoDB та Mongoose: міграція REST API на базу даних

Мета роботи

Метою лабораторної роботи є набуття практичних навичок роботи з MongoDB та Mongoose шляхом міграції REST API з лабораторної роботи №3 з in-memory сховища на повноцінну базу даних, зокрема:

- налаштування MongoDB Atlas та MongoDB Compass для хмарного зберігання та візуального управління даними;
- підключення Mongoose до Express-застосунку з паттерном graceful shutdown;
- проєктування Mongoose-схеми з валідацією, дефолтами та віртуальними властивостями;
- міграція CRUD-операцій з Map-сховища на Mongoose Model;
- централізована обробка специфічних помилок MongoDB (CastError, ValidationError, duplicate key);
- реалізація фільтрації, сортування та пагінації через Mongoose Query API;
- тестування API з використанням mongodb-memory-server замість реальної бази даних.

Хід роботи

Лістинг:

Створення нового файлу src/config/database.ts:

```
import mongoose from 'mongoose';

export async function connectDB(): Promise<void> {
  const uri = process.env.MONGODB_URI;

  if (!uri) {
    console.error('Помилка: MONGODB_URI не знайдено у файлі .env');
    process.exit(1);
  }
}
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.23.000 – Лр.4							
Змн.	Арк.	№ докум.	Підпис	Дата								
Розроб.		Рак І. С.			Звіт з лабораторної роботи №4				Літ.		Арк.	Аркуші
Перевір.		Фуріхата Д. В.									1	10
Реценз.									ФІКТ, гр. ІПЗ-23-2			
Н. Контр.												
Зав.каф.		Вакалюк Т.А.										

```

    try {
      await mongoose.connect(uri);
      console.log('Успішно підключено до MongoDB!');
    } catch (error) {
      console.error('Помилка початкового підключення до MongoDB:', error);
      process.exit(1);
    }
  }

mongoose.connection.on('error', (err) => {
  console.error('MongoDB помилка з\'єднання:', err);
});

mongoose.connection.on('disconnected', () => {
  console.warn('MongoDB відключено');
});

```

Оновлення server.ts:

```

import dotenv from 'dotenv';
dotenv.config();

import app from './app';
import { connectDB } from './config/database';
import mongoose from 'mongoose';

const PORT = process.env.PORT || 3000;

async function startServer() {
  await connectDB();

  const server = app.listen(PORT, () => {
    console.log(`Сервер запущено на http://localhost:${PORT}`);
  });

  process.on('SIGTERM', async () => {
    console.log('Отримано сигнал SIGTERM. Закриваємо HTTP сервер...');
    server.close(async () => {
      console.log('HTTP сервер закрито.');
```

```

      await mongoose.connection.close();
      console.log('З\'єднання з MongoDB закрито.');
```

```

      process.exit(0);
    });
  });
}

startServer();

```

Створення нового файлу src/models/car.model.ts:

```

import { Schema, model, Document } from 'mongoose';

export interface ICar {
  model: string;
  description?: string;
  year: number;
  transmission: 'manual' | 'automatic' | 'robotic';
  isAvailable: boolean;
  createdAt?: Date;
  updatedAt?: Date;
  carAge?: number;
}

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.23.000 – Лр.4	Арк.
						2
Змн.	Арк.	№ докум.	Підпис	Дата		

```

const carSchema = new Schema<ICar>({
  model: {
    type: String,
    required: [true, 'Модель є обов\`язковою'],
    trim: true,
    minlength: 1,
    maxlength: 100
  },
  description: {
    type: String,
    maxlength: 500
  },
  year: {
    type: Number,
    required: true,
    validate: {
      validator: (v: number) => v >= 1886 && v <= new Date().getFullYear(),
      message: (props: any) => `${props.value} не є валідним роком випуску!`
    }
  },
  transmission: {
    type: String,
    enum: ['manual', 'automatic', 'robotic'],
    required: true
  },
  isAvailable: {
    type: Boolean,
    default: true
  }
}, {
  timestamps: true,
  toJSON: { virtuals: true },
  toObject: { virtuals: true }
});

carSchema.virtual('carAge').get(function() {
  return new Date().getFullYear() - this.year;
});

export const CarModel = model<ICar>('Car', carSchema);

```

Оновлення car.ts:

```

import { CarModel, ICar } from '../models/car.model';
import { CreateCarInput, UpdateCarInput } from '../schemas/car.schema';

export class CarStorage {
  async getAll(filter: any, page: number = 1, limit: number = 10) {
    const skip = (page - 1) * limit;

    const [data, total] = await Promise.all([
      CarModel.find(filter).sort({ createdAt: -1 }).skip(skip).limit(limit),
      CarModel.countDocuments(filter)
    ]);

    return {
      data,
      pagination: {
        total,
        page,
        limit,
        pages: Math.ceil(total / limit)
      }
    };
  }
}

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.23.000 – Лр.4	Арк.
						3
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    }
  };
}

async getById(id: string) {
  return await CarModel.findById(id);
}

async create(data: CreateCarInput) {
  return await CarModel.create(data);
}

async update(id: string, data: UpdateCarInput) {
  return await CarModel.findByIdAndUpdate(id, data, {
    returnDocument: 'after',
    runValidators: true
  });
}

async delete(id: string) {
  return await CarModel.findByIdAndDelete(id);
}
}

export const carStorage = new CarStorage();

```

Оновлення файлу errorHandler.ts:

```

import { Request, Response, NextFunction } from 'express';
import { ZodError } from 'zod';
import mongoose from 'mongoose';

export const errorHandler = (err: any, req: Request, res: Response, next:
NextFunction) => {
  if (err instanceof ZodError) {
    return res.status(400).json({
      message: 'Помилка валідації даних (Zod)',
      errors: err.issues
    });
  }

  if (err instanceof mongoose.Error.CastError) {
    return res.status(400).json({ message: 'Невалідний формат ID' });
  }

  if (err instanceof mongoose.Error.ValidationError) {
    const messages = Object.values(err.errors).map(val => val.message);
    return res.status(400).json({
      message: 'Помилка валідації БД',
      errors: messages
    });
  }

  if (err.code === 11000) {
    return res.status(409).json({ message: 'Дублікат унікального ключа (Конфлікт)' });
  }

  console.error('Непередбачувана помилка:', err);
  return res.status(500).json({ message: 'Внутрішня помилка сервера' });
};

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.23.000 – Лр.4	Арк.
						4
Змн.	Арк.	№ докум.	Підпис	Дата		

Створення файлу setup.ts:

```
import { MongoMemoryServer } from 'mongodb-memory-server';
import mongoose from 'mongoose';

let mongoServer: MongoMemoryServer;

export const setupTestDB = async () => {
  mongoServer = await MongoMemoryServer.create();
  const uri = mongoServer.getUri();
  await mongoose.connect(uri);
};

export const clearTestDB = async () => {
  const collections = mongoose.connection.collections;
  for (const key in collections) {
    const collection = collections[key];
    await collection.deleteMany({});
  }
};

export const teardownTestDB = async () => {
  await mongoose.connection.dropDatabase();
  await mongoose.connection.close();
  await mongoServer.stop();
};
```

Оновлення файлу car.test.ts:

```
import request from 'supertest';
import mongoose from 'mongoose';
import app from '../src/app';
import { setupTestDB, clearTestDB, teardownTestDB } from '../setup';
import { CarModel } from '../src/models/car.model';

beforeAll(async () => {
  await setupTestDB();
});

afterEach(async () => {
  await clearTestDB();
});

afterAll(async () => {
  await teardownTestDB();
});

describe('Автомобільний REST API (MongoDB)', () => {

  const validCar = {
    model: 'Toyota Camry',
    description: 'Надійний седан',
    year: 2022,
    transmission: 'automatic',
    isAvailable: true
  };

  describe('Unit-тести Моделі (CarModel)', () => {
    it('повинна коректно зберігати валідний автомобіль', async () => {
      const car = new CarModel(validCar);
      const savedCar = await car.save();
      expect(savedCar._id).toBeDefined();
      expect(savedCar.model).toBe(validCar.model);
    });
  });
});
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.23.000 – Лр.4	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		5

```

        expect(savedCar.createdAt).toBeDefined();
    });

    it('повинна встановлювати дефолтне значення isAvailable = true', async () => {
        const carData = { ...validCar };
        delete (carData as any).isAvailable;

        const car = new CarModel(carData);
        const savedCar = await car.save();
        expect(savedCar.isAvailable).toBe(true);
    });

    it('повинна кидати помилку при невалідному році випуску (кастомний валідатор)', async () => {
        const car = new CarModel({ ...validCar, year: 1800 });
        let err;
        try { await car.save(); } catch (e) { err = e; }
        expect(err).toBeInstanceOf(mongoose.Error.ValidationError);
    });

    it('повинна обчислювати віртуальну властивість carAge', async () => {
        const currentYear = new Date().getFullYear();
        const car = new CarModel({ ...validCar, year: currentYear - 5 });
        expect(car.carAge).toBe(5);
    });
});

describe('Інтеграційні тести API', () => {
    describe('POST /api/cars', () => {
        it('повинен створити новий автомобіль та повернути 201', async () => {
            const res = await request(app)
                .post('/api/cars')
                .send(validCar)
                .expect(201);

            expect(res.body).toHaveProperty('id');
            expect(res.body.model).toBe(validCar.model);
        });

        it('повинен повернути 400 (Zod), якщо відсутні обов'язкові поля', async () => {
            const invalidCar = { year: 2020 };
            const res = await request(app)
                .post('/api/cars')
                .send(invalidCar)
                .expect(400);

            expect(res.body.message).toBe('Помилка валідації даних (Zod)');
        });
    });

    describe('GET /api/cars', () => {
        it('повинен повернути об'єкт з data та pagination, якщо немає записів', async () => {
            const res = await request(app).get('/api/cars').expect(200);
            expect(res.body.data).toEqual([]);
            expect(res.body.pagination.total).toBe(0);
        });

        it('повинен повернути сторінку з автомобілями', async () => {
            await CarModel.create(validCar);
            await CarModel.create({ ...validCar, model: 'Honda Civic' });
        });
    });
});

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.23.000 – Лр.4	Арк.
						6
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        const res = await
request(app).get('/api/cars?limit=1').expect(200);
        expect(res.body.data).toHaveLength(1);
        expect(res.body.pagination.total).toBe(2);
        expect(res.body.pagination.pages).toBe(2);
    });

    describe('GET /api/cars/:id', () => {
        it('повинен повернути автомобіль за існуючим ID', async () => {
            const car = await CarModel.create(validCar);
            const res = await request(app)
                .get(`/api/cars/${car._id}`)
                .expect(200);
            expect(res.body.model).toBe(car.model);
        });

        it('повинен повернути 404, якщо автомобіль не знайдено (але ID валидний)', async () => {
            const fakeId = new mongoose.Types.ObjectId();
            await request(app)
                .get(`/api/cars/${fakeId}`)
                .expect(404);
        });

        it('повинен повернути 400, якщо передано невалидний формат ID (CastError)', async () => {
            const res = await request(app)
                .get('/api/cars/not-a-mongo-id')
                .expect(400);
            expect(res.body.message).toBe('Невалидний формат ID');
        });
    });

    describe('PATCH /api/cars/:id', () => {
        it('повинен частково оновити автомобіль та повернути 200', async () => {
            const car = await CarModel.create(validCar);
            const res = await request(app)
                .patch(`/api/cars/${car._id}`)
                .send({ year: 2025 })
                .expect(200);

            expect(res.body.year).toBe(2025);
            expect(res.body.model).toBe(validCar.model);
        });

        it('повинен повернути 400 (Помилка БД) при спробі зберегти дані, що порушують схему', async () => {
            const car = await CarModel.create(validCar);

            let err;
            try {
                await CarModel.findByIdAndUpdate(car._id, { transmission: 'invalid_enum' }, { runValidators: true });
            } catch (e) { err = e; }
            expect(err).toBeInstanceOf(mongoose.Error.ValidationError);
        });
    });

    describe('DELETE /api/cars/:id', () => {
        it('повинен видалити автомобіль та повернути 204', async () => {
            const car = await CarModel.create(validCar);

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.23.000 – Лр.4	Арк.
						7
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        await request(app)
            .delete(`/api/cars/${car._id}`)
            .expect(204);

        const foundCar = await CarModel.findById(car._id);
        expect(foundCar).toBeNull();
    });

    describe('GET /api/cars/available (Специфічний маршрут)', () => {
        it('повинен повертати лише доступні автомобілі', async () => {
            await CarModel.create({ ...validCar, isAvailable: true });
            await CarModel.create({ ...validCar, isAvailable: false });

            const res = await
request(app).get('/api/cars/available').expect(200);
            expect(res.body).toHaveLength(1);
            expect(res.body[0].isAvailable).toBe(true);
        });
    });

    describe('Обробник помилок (500)', () => {
        it('повинен повернути статус 500, якщо сталася внутрішня помилка сервера', async () => {
            jest.spyOn(CarModel, 'find').mockImplementationOnce(() => {
                throw new Error('Системний збій БД');
            });

            const res = await request(app)
                .get('/api/cars')
                .expect(500);

            expect(res.body.message).toBe('Внутрішня помилка сервера');
        });
    });

    describe('Граничні випадки та помилки БД (для 100% покриття)', () => {
        it('повинен повернути 409 при дублюванні унікального ключа (код 11000)',
async () => {
            jest.spyOn(CarModel, 'create').mockImplementationOnce(() => {
                throw { code: 11000 }; // Імітуємо конфлікт
            });
            await request(app).post('/api/cars').send(validCar).expect(409);
        });

        it('повинен ловити помилки в catch для GET /available', async () => {
            jest.spyOn(CarModel, 'find').mockImplementationOnce(() => { throw new
Error('DB'); });
            await request(app).get('/api/cars/available').expect(500);
        });

        it('повинен ловити помилки в catch для PATCH', async () => {
            const fakeId = new mongoose.Types.ObjectId();
            jest.spyOn(CarModel, 'findByIdAndUpdate').mockImplementationOnce(() =>
{ throw new Error('DB'); });
            await request(app).patch(`/api/cars/${fakeId}`).send({ year: 2025
}).expect(500);
        });

        it('повинен ловити помилки в catch для DELETE', async () => {
            const fakeId = new mongoose.Types.ObjectId();
            jest.spyOn(CarModel, 'findByIdAndDelete').mockImplementationOnce(() =>
{ throw new Error('DB'); });

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.23.000 – Лр.4	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		8


```

    await request(app).delete(`/api/cars/${fakeId}`).expect(500);
  });

  it('повинен гарантовано ловити CastError', async () => {
    const castErr = new mongoose.Error.CastError('ObjectId', 'fake',
'id');
    jest.spyOn(CarModel, 'findById').mockImplementationOnce(() => { throw
castErr; });
    await request(app).get('/api/cars/fake').expect(400);
  });
});
});
});

```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	94.21	75	95.23	94.78	
src	100	100	100	100	
app.ts	100	100	100	100	
src/middleware	86.95	90	75	90	
errorHandler.ts	81.25	90	50	85.71	18-19
validate.ts	100	100	100	100	
src/models	100	100	100	100	
car.model.ts	100	100	100	100	
src/routes	92	71.42	100	92	
car.ts	92	71.42	100	92	25,28,69,82
src/schemas	100	100	100	100	
car.schema.ts	100	100	100	100	
src/storage	100	0	100	100	
car.ts	100	0	100	100	5
tests	100	100	100	100	
setup.ts	100	100	100	100	
Test Suites: 1 passed, 1 total					
Tests: 21 passed, 21 total					
Snapshots: 0 total					
Time: 2.395 s					

Рис.1 Зображення результату

Висновок:

У ході виконання лабораторної роботи №4 було успішно здійснено міграцію REST API з тимчасового in-memoгу сховища на повноцінну документно-орієнтовану базу даних MongoDB .

Основними результатами роботи є:

- **Налаштування хмарного середовища:** Створено кластер у MongoDB Atlas, налаштовано мережевий доступ та здійснено підключення до бази даних за

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.23.000 – Лр.4	Арк.
						9
Змн.	Арк.	№ докум.	Підпис	Дата		

допомогою візуального клієнта MongoDB Compass для зручного керування документами .

- **Інтеграція Mongoose:** Налаштовано підключення Express-застосунку до MongoDB за допомогою Mongoose ODM. Впроваджено патерн graceful shutdown для коректного закриття з'єднань із базою даних при завершенні роботи сервера .
- **Проектування схеми даних:** Створено Mongoose-схему для сутності автомобіля, яка працює в синергії з Zod-валідацією. Реалізовано кастомні валідатори, значення за замовчуванням та віртуальні властивості (обчислюване поле carAge), які автоматично включаються у JSON-відповідь.
- **Міграція CRUD та Query API:** Всі операції зі сховищем переведено на використання методів Mongoose Model (find, create, findByIdAndUpdate тощо). Повністю перероблено механізм фільтрації на базі Mongoose Query API, а також успішно додано сортування та пагінацію результатів .
- **Розширена обробка помилок:** Централізований errorHandler було адаптовано для перехоплення та обробки специфічних помилок бази даних, таких як невалідний формат ідентифікатора (CastError), порушення правил схеми (ValidationError) та конфлікт унікальності (Duplicate Key 11000) .
- **Ізольоване тестування:** Проведено рефакторинг тестової бази. Завдяки використанню пакета mongodb-memory-server, інтеграційні та unit-тести тепер виконуються у повноцінній віртуальній MongoDB в оперативній пам'яті, що забезпечує їхню швидкість та ізолюваність . Код покритий тестами майже на 95%.

<https://git.ztu.edu.ua/node-js/lab3>

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.23.000 – Лр.4	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		10